

Chapter 9. Gate A-G가 Book-forge에서 실제로 측정하는 것

이 챕터에서 배우는 것

- Tracker·Config·Gate — 지금까지 뒤섞어 써온 세 용어가 실제로는 서로 다른 층이라는 것
- 7개 Gate(A-G) 중 Book-forge가 실제로 값을 채우는 것은 몇 개인지
- 왜 어떤 Gate는 자주 "N/A"로 나오는지
- 한 챕터를 실제로 채점해보면 어떤 숫자가 나오는지

이런 분이 먼저 읽으면 좋습니다: "58개 지표, 7개 Gate"라는 Agent-Evaluator SDK의 전체 규모에 압도됐지만, 실제 프로젝트 하나가 그중 정확히 무엇을 쓰는지 궁금한 분.

9.1 Tracker · Config · Gate — 0장에서 본 세 층을 코드로 증명한다

0장(§ 0.4)이 이미 Tracker·Config·Gate 세 층의 정의와 "Config를 하나도 안 켜도 Tracker는 이미 켜져 있다"는 결론을 미리 보여줬다 — 이 절은 그 주장을 뒷받침하는 **실제 코드**를 처음으로 연다. 아직 0장을 안 읽었다면 세 층의 정의표부터 확인하고 오는 편이 낫다.

`eval/monitor.py`의 `build_book_monitor()`가 `PerformanceMonitor`를 만드는 순간, SDK 내부에서는 이미 이런 일이 일어난다(Agent-Evaluator SDK의 `core/trackers/monitor.py` 실제 초기화 코드) — 이 책 어느 챕터도 지금까지 이 초기화 코드를 직접 보여준 적이 없다.

```
# PerformanceMonitor.__init__() 내부 — 에이전트가 무엇을 하든 상관없이 항상 실행된다
self.tcr_tracker = TaskCompletionTracker()
self.latency_tracker = LatencyTracker()
```

```
self.token_tracker = TokenEconomyTracker(pricing)
self.agent_coordination_tracker = AgentCoordinationTracker()
```

`ChatAgent` (7장)의 `@agent_eval`에는 `SLAConfig`만 있고 `AgentRoleConfig`는 없지만, `agent_coordination_tracker`는 여전히 존재한다 — 다만 아무도 `track_interaction()`을 호출하지 않았을 뿐이다(5장의 리뷰 패널만 이 메서드를 부른다). **Config는 "이 Tracker를 켜다"가 아니라 "이미 켜져 있는 Tracker의 판정 기준을 조정한다"에 가깝다.**

Tracker 하나가 실제로 어떻게 동작하는지 `LatencyTracker`로 확인해보면 이 층의 정체가 뚜렷해진다.

```
class LatencyTracker(BaseTracker):
    def record_latency(self, task_id: str, task_type: str,
                      total_time: float, breakdown: dict[str, float]):
        """Record latency for a task"""
        self._latencies.append({
            "task_id": task_id, "task_type": task_type,
            "total_time": total_time, "breakdown": breakdown,
        })

    def get_latency_stats(self, task_type: str | None = None) -> dict[str, float]:
        """p50/p95/p99 등 지연 통계를 계산해 돌려준다."""
        ...
```

`record_latency()`가 매 호출마다 숫자를 쌓고(수집), `get_latency_stats()`가 그 누적치에서 p95·p99 같은 통계를 뽑는다(집계) — **모든 Tracker가 이 "쌓기 → 계산하기" 두 메서드 쌍 패턴을 공유한다.** `TaskCompletionTracker`도 마찬가지다 — `add_task()`가 태스크 결과를 쌓고, `calculate_tcr()`이 그중 완료율(TCR)을 계산한다. Gate가 최종적으로 읽는 것은 원시 데이터가 아니라 이 `get_*_stats()` / `calculate_*()` 메서드들이 이미 계산해 둔 값이다.

`SLAConfig`가 실제로 Tracker의 결과를 어떻게 조정하는지도 정의를 보면 분명해진다.

```
@dataclass
class SLAConfig:
    """SLA 준수 추적 설정."""
    p95_ms: float = 5000.0
    p99_ms: float = 10000.0
```

```
breach_window: int = 10
warn_threshold: int = 2
fail_threshold: int = 5
max_cost_per_task: float | None = None
```

`ChapterDrafterAgent` (8장)가 `SLAConfig(p95_ms=60_000, p99_ms=90_000)`를 넘기는 것은, `LatencyTracker`가 이미 계산해 둔 p95·p99 값을 "이 숫자를 넘으면 위반"이라는 기준과 대조하라고 Gate D에 알려주는 것이다 — `LatencyTracker` 자신은 `SLAConfig`가 있는지 없는지조차 모른다. **Tracker는 "무슨 일이 있었는가"를 객관적으로 기록하고, Config는 "그게 괜찮은 일이었는가"를 판단할 기준선을 정하고, Gate는 그 판단들을 모아 하나의 점수로 압축한다** — 이 세 층의 역할 분리가 8~11장 전체를 관통하는 뼈대다.

9.2 33개 Config·25개 트래커가 아니라, Book-forge가 실제로 쓰는 것만

Agent-Evaluator SDK 전체는 25개 네이티브 트래커와 33개 Harness Config를 제공한다. 이 챕터는 그 전체 카탈로그를 나열하지 않는다 — 8장에서 확인한 Book-forge의 실제 에이전트 14개가 **실제로 건드리는** Gate만 정리한다.

Gate	Book-forge가 채우는 값	어느 에이전트가 기여하는가
A 목표 달성	TaskCompletionTracker + AccuracyEvaluator 블렌딩 ($0.4 \times \text{TCR} + 0.6 \times \text{Accuracy}$)	전체 14개 — <code>ground_truth</code> 를 넘기는 모든 에이전트
B 행동 무결성	LoopDetectionConfig 기반 반복 탐지	<code>revise()</code> (<code>review_loop.py</code>)만 — 나머지는 대부분 N/A
C 신뢰성	HallucinationDetector(RAG 근거 대조)	<code>rag_mode=True</code> 인 6개 (ChapterDrafter·ReferenceTable·Diagram·Capstone·ModuleReference·Chat)
D 성능 계약	LatencyTracker(항상) + SLAConfig 위반 여부	전체(LatencyTracker는 항상 기록)
E 보안 경계	5개 보안 트래커 + ThreatSeverityConfig	위 6개(<code>rag_mode=True</code>)가 명시적, 나머지는 <code>enable_security_metrics=True</code> 로 상시 on

Gate	Book-forge가 채우는 값	어느 에이전트가 기여하는가
F 다중 에이전트	AgentCoordinationTracker + ConflictResolutionConfig	ReviewPanel(5장)만 — 유일하게 값이 나오는 지점
G 관측성	ExplainabilityConfig	PlannerAgent·AlternativeSuggesterAgent·SlideCondenserAgent 3개가 명시적, 나머지는 대부분 N/A

9.3 "N/A"가 버그가 아니라 정직한 신호인 이유

이 표를 처음 보면 Gate B·F·G가 유독 자주 비어 보인다는 인상을 받을 수 있다. 이걸 계측 누락이 아니라 **그 Gate가 실제로 측정할 신호 자체가 그 태스크에 없기 때문**이다.

- Gate F(다중 에이전트)는 `AgentCoordinationTracker`가 `track_interaction()` 호출을 필요로 한다(5장 § 5.3). PlannerAgent 하나만 도는 태스크에는 애초에 "다른 에이전트와의 상호작용"이라는 사건 자체가 없다 — N/A가 정직한 답이다.
- Gate B(행동 무결성)의 루프 탐지도 마찬가지로 — 한 번만 호출되는 함수에는 "반복"이라는 개념이 성립하지 않는다.

`monitor.py` (agent-evaluator SDK)의 원칙이 정확히 이렇다 — 데이터가 없는 Gate는 억지로 기본값(예: 1.0이나 0.5)을 채우지 않고 N/A로 남긴다. **거짓으로 만점을 주는 것보다 "측정하지 않았다"고 정직하게 말하는 편이, 이후 그 Gate 점수를 신뢰할지 판단하는 사람에게 더 유용하다.**

9.4 Gate A의 실제 계산 — TCR과 Accuracy를 섞는다

Gate A는 이 책이 가장 자주 언급하는 Gate다 (PlannerAgent·TOCDesignerAgent·ChapterDrafterAgent 전부 기여). 실제 점수 계산은 다음 공식을 따른다(`CLAUDE.md` 에 문서화된 Book-forge 전역 설정).

```
_a_score = gate_a_tcr_weight × TCR컴포넌트 + (1 - gate_a_tcr_weight) × 나머지 평균
```

기본값 `gate_a_tcr_weight=0.4` — 태스크 완료율(TCR)이 40%, 나머지 (AccuracyEvaluator 블렌딩·ResponseQualityEvaluator 등)가 60%를 차지한다. 14장에서 이 가중치를 프로젝트마다 다르게 조정하는 실제 코드(`.env`)를 다룬다.

9.5 실제 채점 사례 — 한 챕터의 Gate 점수

이 책을 준비하며 실제로 `book-forge new --source ...` 로 챕터 하나를 생성했을 때 나온 실제 로그다(이 세션의 실제 실행 기록).

이 초안의 Gate 점수 (참고용 — 전체 판정은 book-forge gate로):

```
⚠ A (목표 달성): 0.663
  · B (행동 무결성): N/A
✅ C (신뢰성): 0.708
❌ D (성능 계약): 0.135
✅ E (보안 경계): 1.000
  · F (다중 에이전트): N/A
⚠ G (관측성): 0.694
```

이 예시가 § 9.3의 주장을 그대로 뒷받침한다 — B·F는 N/A(그 태스크에 반복이나 다중 에이전트 상호작용이 없었으므로), D는 0.135로 낮게 나왔다(로컬 35B 모델이 SLA 60초 기준을 자주 초과했기 때문 — 로컬 추론의 실제 트레이드오프다). E는 1.000인데, 이는 "보안이 완벽하다"는 뜻이라기보다 이 특정 태스크에서 5개 보안 트래커가 위반을 하지도 못 했었다는 뜻이다 — Gate 점수는 항상 **그 태스크에서 실제로 검사한 것에 한해서만** 유효하다.

이 아이콘(✅/⚠/❌/·)이 어디서 나오는지도 코드로 확인할 수 있다 —

`eval/gate_summary.py` 의 `format_gate_line()` 이 한 Gate의 점수 하나를 받아 이 형식의 문자열 한 줄로 바꾼다.

```
def format_gate_line(gate: str, score: Optional[float]) -> str:
    label = GATE_LABELS.get(gate, gate)
    if score is None:
        return f"  · {gate} ({label}): N/A"
    if score >= 0.7:
        icon = "✅"
    elif score >= 0.5:
        icon = "⚠"
    else:
        icon = "❌"
```

```
icon = "❌"  
return f" {icon} {gate} ({label}): {score:.3f}"
```

기준선은 단순하다 — 0.7 이상은 , 0.5~0.7은 , 0.5 미만은 , `None` (N/A)은 . 이 함수는 Gate 점수를 계산하지 않는다 — 이미 계산된 `score` 를 사람이 한눈에 읽을 수 있는 표시로만 바꾼다(계산과 표시를 분리하는 흔한 패턴). `draft_cmd.py` 가 챗터 하나를 생성한 직후 이 함수를 Gate A~G 각각에 대해 호출해, 위에서 본 실제 로그를 만든다.

 **N/A와 낮은 점수를 혼동하지 말 것:** `N/A` 는 "측정 대상이 아니었다"이고, `0.135` (D)는 "측정했고, 기준을 못 넘었다"다. 이 둘을 같은 것으로 취급해 평균에 섞으면 안 된다 — 실제로 `HarnessEvaluationGate` 는 데이터 없는 Gate를 평균에서 제외한다.

직접 해보기

0장 (§ 0.6)에서 실행한 `book-forge gate` 의 실제 출력을 다시 열어, § 9.1의 Tracker·Config·Gate 표와 나란히 놓고 대조해보라 — 화면에 찍힌 각 줄(`A (목표 달성): 0.663` 등)이 어떤 Tracker의 계산값이고, 어떤 Config가 그 판정 기준을 조정했는지 하나씩 짚어낼 수 있다면 이 챗터를 제대로 소화한 것이다. N/A로 나온 Gate가 있다면 왜 N/A인지 (§ 9.3)도 함께 설명해보라 — "그 Gate가 측정할 신호 자체가 이 테스트에 없었다"는 답이 나와야 한다.

이 챗터의 핵심

- **Tracker(항상 켜짐, 데이터 수집·계산) → Config(에이전트별 판정 기준 조정, 옵션) → Gate(A-G 최종 점수, 집계)는 서로 다른 층이다.** Config를 하나도 안 써도 Tracker는 이미 데이터를 쌓고 있다.
- **Book-forge는 7개 Gate 중 정확히 필요한 신호만 채운다.** 나머지는 억지로 채우지 않고 N/A로 남긴다.

- **N/A는 결함이 아니라 정직함이다.** "이 태스크에는 이 Gate가 측정할 신호 자체가 없었다"는 뜻이다.
- **Gate A는 TCR과 Accuracy를 가중 평균한다(기본 0.4:0.6).** 이 가중치는 조정 가능하다(14장).
- **실측 사례에서 Gate D(성능)가 자주 낮게 나온다.** 로컬 모델의 지연 시간이 SLA 기준(60초)을 넘기는 경우가 실제로 있다.

참고 자료

- 부록 B.1·B.2(업계 동향) — 환각 탐지·에이전트 관측성이 업계 전체에서는 어떤 규모·이름으로 다뤄지는지
- `Agent-Evaluator/CLAUDE.md` — Gate별 트래커 기여 방식 전체 표
- `Agent-Evaluator/agent_evaluator/core/trackers/monitor.py` — `PerformanceMonitor.__init__()`의 Tracker 초기화, `layer1.py` / `layer2.py` / `security.py`의 개별 Tracker 클래스
- `Agent-Evaluator/agent_evaluator/gates/gate_d_performance/configs.py` — `SLAConfig`
- `src/book_forge/eval/monitor.py` — `build_book_monitor()`
- `src/book_forge/eval/gate_summary.py` — `format_gate_line()`, `GATE_LABELS`
- `src/book_forge/cli/commands/draft_cmd.py` — 챗터 생성 직후 Gate 점수를 출력하는 코드

다음 챗터는 Gate A-G가 손대지 않는 영역 — LLM을 아예 호출하지 않는 순수 정적 검증기 3종을 다룬다.